

DepthWizard

Single-View Height Estimation and Interactive 3D Flythrough

TEAM TECHNICAL UNDERSTANDING & JUDGE PREPARATION GUIDE

A beginner-friendly guide to the complete six-person pipeline: image ingestion, monocular relative depth, geospatial calibration, API integration, web experience, and GPU-accelerated 3D terrain.

Edition	Verified against	Audience
5 September 2026	Current local repository and test suite	All six teammates and hackathon judges

CORE PROMISE DepthWizard turns one RGB image into an explorable height surface. It reports relative height when no trustworthy elevation reference exists and estimated metric elevation only after valid georeferencing plus SRTM and/or GCP calibration.

Hackathon prototype · not survey-grade photogrammetry · not a LiDAR replacement

How to use this guide

This is both a learning document and a judging-day revision pack. Read Chapters 1–4 once as a team, then study your own person chapter. Before the demo, every teammate should review the limitations, the end-to-end data contract, the judge Q&A, and their one-page cheat sheet.

Reading path	Best for	Sections
15-minute team overview	Understanding the product story	Executive summary, Chapters 1–4, limitations
Role deep dive	Preparing your own technical explanation	Your person chapter + your cheat sheet
Demo operator	Running and troubleshooting the live system	Integration, demo runbook, startup troubleshooting
Judge preparation	Handling critical questions honestly	Validation, limitations, 40 suggested answers

Meaning of status labels

Label	Meaning in this document
Implemented	The behavior exists in the current repository and is exercised by source code or tests.
Optional	Supported only when the user provides an additional file or configuration.
Prototype limitation	A known boundary that must be explained rather than hidden.
Future improvement	A sensible next step; it is not claimed as current functionality.

Words we use carefully

- Relative depth: a scene-dependent ordering or geometry signal with no physical unit.
- Relative height: the same unitless signal displayed as vertical relief; the viewer labels it rel.
- Estimated absolute DSM: a reference-calibrated surface in metres, still subject to model, alignment, datum, and reference error.
- Ground height: not automatically available. Building or canopy height above ground requires a trustworthy ground DTM and subtraction.

Contents

The final PDF contains a generated table of contents with page numbers and clickable heading bookmarks.

- How to use this guide 2**
 - Meaning of status labels 2
 - Words we use carefully 2
- Contents 3**
- Executive summary 9**
 - The two honest operating modes 9
 - What is currently implemented 10
- 1 • Problem definition and why it matters 11**
 - Polished hackathon problem statement 11
 - Why the problem is hard 11
 - Responsible application areas 11
- 2 • Concepts every teammate should understand 12**
 - RGB image, raster, and height surface 12
 - DEM, DTM, DSM, and nDSM 12
 - Relative depth versus absolute elevation 12
 - Does the model know roads, buildings, and canopy? 12
- 3 • Geospatial concepts without the jargon 13**
 - GeoTIFF is more than a TIFF file 13
 - Why ordinary image resizing is not reprojection 13
- 4 • Intended architecture versus verified implementation 14**
 - The intended conceptual flow 14
 - What the repository actually does 14
- 5 • Person 1 — Image loading, GeoTIFF metadata, and preprocessing 15**
 - What this module does from scratch 15
 - Bands and array orientation 15
 - How a pixel retains its Earth location 15
 - Actual files 15
 - Input and output contract 16
 - Common failure cases 16
 - Person 1 — 30-second explanation 16
- 6 • Person 2 — Monocular relative depth estimation 17**
 - What monocular depth means 17
 - The model actually used 17
 - Large-scene inference 17
 - Numerical output versus grayscale preview 18
 - Why Person 3 is still necessary 18
 - Actual files 18
 - Failure cases and responsible answers 18
 - Person 2 — 30-second explanation 19
- 7 • Person 3 — SRTM/GCP calibration and DSM generation 20**

Why SRTM is useful—but not enough	20
What a GCP contributes	20
CRS-aware alignment	20
Calibration candidates	20
Fusion	21
SRTM-only calibration	21
Output and validation	21
Vertical datum caution	22
What happens when calibration is absent	22
Person 3 — 30-second explanation	22
8 • Person 4 — Frontend website and user experience	23
Actual stack	23
User flow	23
Communicating uncertainty	23
Actual files	23
Why heavy computation stays out of the browser	24
Person 4 — 30-second explanation	24
9 • Person 5 — Backend and API integration	25
Backend and API basics	25
Actual stack and endpoints	25
Stage ordering and progress	25
Data and file safety	25
Relative versus calibrated branch	26
Result contract	26
Prototype scaling boundary	26
Person 5 — 30-second explanation	26
10 • Person 6 — 3D terrain rendering and flythrough	27
Height map to mesh	27
Physical versus relative scaling	27
Rendering quality features in the current code	27
Texture and UV mapping	28
Controls that actually exist	28
How point values are calculated	28
GPU use	28
Person 6 — 30-second explanation	29
11 • Complete system integration	30
End-to-end data flow	30
The non-georeferenced path	30
The calibrated GeoTIFF path	30
Why dimensions are a team-wide contract	30
What is downloaded versus displayed	30
12 • Validation, evaluation, and what good evidence looks like	31
Current automated verification	31
Accuracy study the team should run	31

How to validate the 3D experience	31
13 • Important limitations and assumptions	32
What the team can confidently claim	32
14 • Demo runbook and troubleshooting	33
Fastest reliable startup on Windows	33
If using PowerShell scripts	33
Pre-demo checklist	33
Common startup problems	33
15 • Possible judge questions and suggested answers	34
Cheat sheet • Person 1 — Image loading, GeoTIFF metadata, and preprocessing	42
30-second explanation	42
2-minute detailed explanation	42
Cheat sheet • Person 2 — Monocular relative depth estimation	43
30-second explanation	43
2-minute detailed explanation	43
Cheat sheet • Person 3 — SRTM/GCP calibration and DSM generation	44
30-second explanation	44
2-minute detailed explanation	44
Cheat sheet • Person 4 — Frontend website and user experience	45
30-second explanation	45
2-minute detailed explanation	45
Cheat sheet • Person 5 — Backend and API integration	46
30-second explanation	46
2-minute detailed explanation	46
Cheat sheet • Person 6 — 3D terrain rendering and interactive flythrough	47
30-second explanation	47
2-minute detailed explanation	47
Appendix A • Glossary	48
Appendix B • Verified repository file map	49
Default configuration snapshot	49
Appendix C • References and further reading	50
Final team message	50
Meaning of status labels 2	
Words we use carefully 2	
Contents 3	
Executive summary 6	
The two honest operating modes 6	
What is currently implemented 6	
1 • Problem definition and why it matters 8	
Polished hackathon problem statement 8	
Why the problem is hard 8	
Responsible application areas 8	
2 • Concepts every teammate should understand 9	

RGB image, raster, and height surface 9

DEM, DTM, DSM, and nDSM 9

Relative depth versus absolute elevation 9

Does the model know roads, buildings, and canopy? 9

3 · Geospatial concepts without the jargon 10

GeoTIFF is more than a TIFF file 10

Why ordinary image resizing is not reprojection 10

4 · Intended architecture versus verified implementation 11

The intended conceptual flow 11

What the repository actually does 11

5 · Person 1 — Image loading, GeoTIFF metadata, and preprocessing 12

What this module does from scratch 12

Bands and array orientation 12

How a pixel retains its Earth location 12

Actual files 12

Input and output contract 13

Common failure cases 13

Person 1 — 30-second explanation 13

6 · Person 2 — Monocular relative depth estimation 14

What monocular depth means 14

The model actually used 14

Large-scene inference 14

Numerical output versus grayscale preview 15

Why Person 3 is still necessary 15

Actual files 15

Failure cases and responsible answers 16

Person 2 — 30-second explanation 16

7 · Person 3 — SRTM/GCP calibration and DSM generation 17

Why SRTM is useful—but not enough 17

What a GCP contributes 17

CRS-aware alignment 17

Calibration candidates 17

Fusion 18

SRTM-only calibration 18

Output and validation 18

Vertical datum caution 19

What happens when calibration is absent 19

Person 3 — 30-second explanation 19

8 · Person 4 — Frontend website and user experience 20

Actual stack 20

User flow 20

Communicating uncertainty 20

Actual files 20

Why heavy computation stays out of the browser 21

Person 4 — 30-second explanation 21

9 · Person 5 — Backend and API integration 22

Backend and API basics 22

Actual stack and endpoints 22

Stage ordering and progress 22

Data and file safety 22

Relative versus calibrated branch 23

Result contract 23

Prototype scaling boundary 23

Person 5 — 30-second explanation 23

10 · Person 6 — 3D terrain rendering and flythrough 24

Height map to mesh 24

Physical versus relative scaling 24

Rendering quality features in the current code 24

Texture and UV mapping 25

Controls that actually exist 25

How point values are calculated 25

GPU use 25

Person 6 — 30-second explanation 26

11 · Complete system integration 27

End-to-end data flow 27

The non-georeferenced path 27

The calibrated GeoTIFF path 27

Why dimensions are a team-wide contract 27

What is downloaded versus displayed 27

12 · Validation, evaluation, and what good evidence looks like 28

Current automated verification 28

Accuracy study the team should run 28

How to validate the 3D experience 28

13 · Important limitations and assumptions 29

What the team can confidently claim 29

14 · Demo runbook and troubleshooting 30

Fastest reliable startup on Windows 30

If using PowerShell scripts 30

Pre-demo checklist 30

Common startup problems 30

15 · Possible judge questions and suggested answers 31

Cheat sheet · Person 1 — Image loading, GeoTIFF metadata, and preprocessing 39

30-second explanation 39

2-minute detailed explanation 39

Cheat sheet · Person 2 — Monocular relative depth estimation	40
30-second explanation	40
2-minute detailed explanation	40
Cheat sheet · Person 3 — SRTM/GCP calibration and DSM generation	41
30-second explanation	41
2-minute detailed explanation	41
Cheat sheet · Person 4 — Frontend website and user experience	42
30-second explanation	42
2-minute detailed explanation	42
Cheat sheet · Person 5 — Backend and API integration	43
30-second explanation	43
2-minute detailed explanation	43
Cheat sheet · Person 6 — 3D terrain rendering and interactive flythrough	44
30-second explanation	44
2-minute detailed explanation	44
Appendix A · Glossary	45
Appendix B · Verified repository file map	46
Default configuration snapshot	46
Appendix C · References and further reading	47
Final team message	47

Executive summary

Generating useful 3D elevation information normally depends on stereo imagery, LiDAR, photogrammetry, radar interferometry, or an existing elevation product. DepthWizard explores a faster, lower-input route: infer relative scene structure from one optical image, anchor it to real-world elevation when suitable references exist, and turn the result into an interactive 3D terrain that a user can inspect immediately.

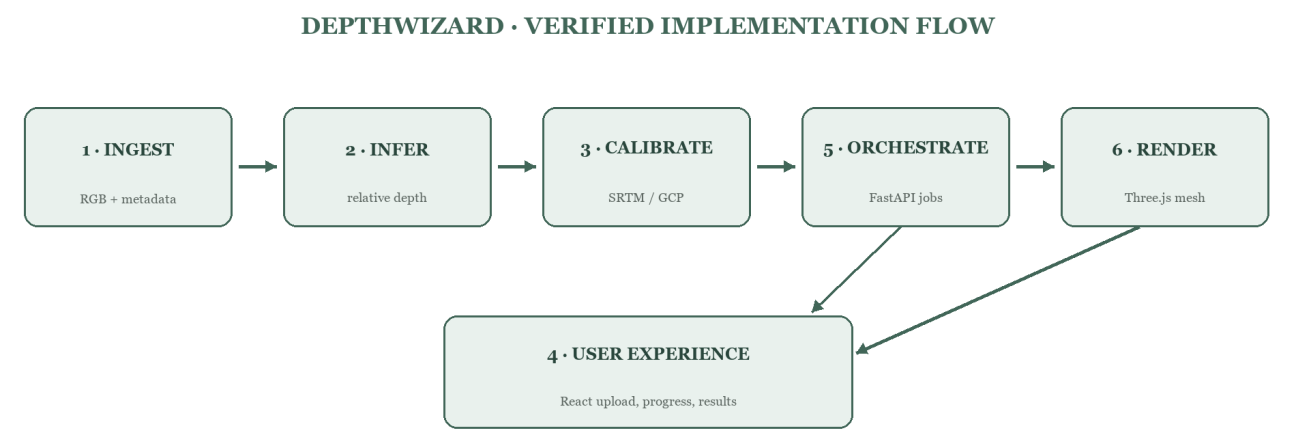


Figure 1. Current repository architecture. Person 5 orchestrates the processing modules while Person 4 presents Person 6’s viewer.

The two honest operating modes

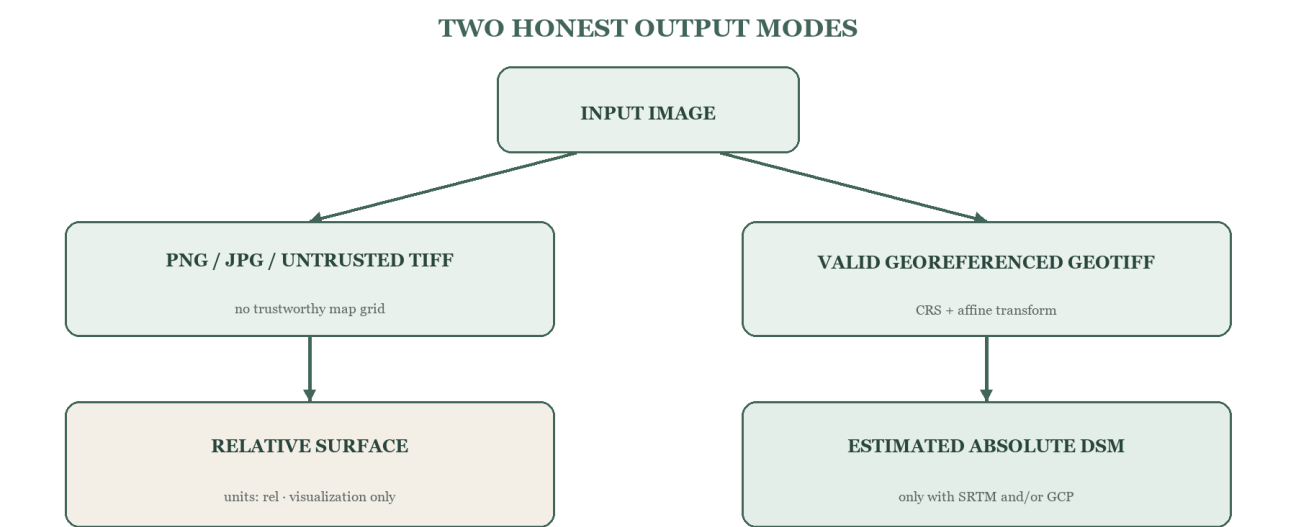


Figure 2. A file extension alone does not decide the output mode; trustworthy metadata and calibration inputs do.

THE ONE SENTENCE EVERYONE SHOULD REMEMBER A single image can provide useful relative structure, but it cannot by itself establish exact elevation above sea level. DepthWizard adds absolute reference only when geospatial metadata and SRTM/GCP evidence make that possible.

What is currently implemented

Capability	Current implementation	Status
Input	PNG, JPG/JPEG, TIFF/GeoTIFF; bounded 3072-pixel working grid for large scenes	Implemented
Relative depth	Depth Anything V2 Base by default; CUDA when available; global plus overlapping 768 px tiles	Implemented
Absolute calibration	Local SRTM/elevation raster and/or CSV/JSON GCPs; linear, inverse, robust-linear candidates	Optional
Geospatial output	Float32 absolute_dsm.tif plus NPY, preview, metrics, and metadata for calibrated jobs	Implemented
Web application	React frontend built with Vite; FastAPI backend with background jobs and polling	Implemented
3D	Three.js/WebGL terrain, texture/elevation/wireframe modes, overview, free flight, auto path, point HUD	Implemented
Standalone scale	Single-process local hackathon workflow; no durable worker queue or cloud autoscaling	Prototype limitation

1 • Problem definition and why it matters

Polished hackathon problem statement

High-quality elevation and 3D surface data are valuable, but the usual acquisition routes—LiDAR, stereo photogrammetry, interferometric radar, or commercial DEM products—may be unavailable, expensive, slow, or operationally inconvenient. DepthWizard investigates whether a single RGB remote-sensing image, combined when possible with coarse elevation references and control points, can produce an approximate surface model that is immediately understandable through an interactive flythrough.

The goal is not to defeat the laws of geometry or replace survey instruments. The goal is to create a practical, transparent pipeline that gives useful relative structure from ordinary imagery, upgrades that structure to an estimated metric surface when valid references exist, and makes uncertainty visible to the user.

Why the problem is hard

Difficulty	Plain-language explanation	DepthWizard response
One view is ambiguous	Different 3D scenes can create a similar 2D picture.	Use a pretrained prior and label the result as an estimate.
No automatic metres	Model values do not contain sea-level elevation or a universal scale.	Fit scale/offset from SRTM or GCPs when available.
Domain gap	Models trained on natural perspective images may misread nadir aerial scenes.	Use a stronger relative model, tiles, cautious smoothing, and explicit limitations.
Geospatial grids differ	SRTM and optical pixels may use different coordinates and resolutions.	Reproject with CRS and affine transforms before comparing pixels.
3D can mislead	A beautiful textured mesh can look correct even when geometry is wrong.	Keep relative/metric labels, source metadata, and raw measurement values visible.

Responsible application areas

- Rapid visual familiarization with a remote area when richer 3D data is not immediately available.
- Educational demonstrations of DSMs, CRSs, monocular depth, and 3D terrain generation.
- Early-stage planning or communication for infrastructure and landform discussions.
- Disaster-response research prototypes for situational visualization—only after independent validation before operational use.
- Remote-sensing and defence research demonstrations where uncertainty and provenance are preserved.

CONFIDENCE, NOT HYPE The strongest pitch is that DepthWizard integrates six difficult layers into one transparent workflow. Its value is accessibility, speed, and inspectability—not a claim of survey-grade elevation from one image.

2 • Concepts every teammate should understand

RGB image, raster, and height surface

An RGB image stores three brightness values per pixel: red, green, and blue. A raster is simply an organized grid of cells. It is not “one raster per pixel.” One raster has rows and columns, and each band stores a value at every cell. A height map is another raster: each cell stores one height-like value instead of a color.

Data	Value at one cell	Units	Typical use
RGB image	Red, green, blue brightness	Usually 0–255 after display normalization	Appearance / texture
Relative depth	Learned near/far score	Unitless	Relative shape
SRTM/DEM	Reference elevation	Usually metres for the selected product	Broad absolute terrain
DSM	Top-surface elevation	Metres when calibrated	Terrain + structures/canopy surface
nDSM	DSM – reliable ground DTM	Metres	Height above ground

DEM, DTM, DSM, and nDSM

DEM is a broad family term for elevation grids. DTM usually means a bare-earth terrain surface. DSM means the upper visible or sensed surface, which may include roofs and vegetation. nDSM means normalized DSM: the height above a ground surface. Real products use these terms inconsistently, so the safest description of DepthWizard’s calibrated result is “estimated/fused DSM or height surface.”

nDSM = estimated DSM – trustworthy ground DTM

Relative depth versus absolute elevation

Suppose two pixels receive model values 0.8 and 0.2. That may tell us that one appears closer to the camera, but it does not mean 0.8 m and 0.2 m, or 80 m and 20 m. Absolute elevation needs a reference with known height and a compatible coordinate system.

Does the model know roads, buildings, and canopy?

Not as an explicit rule. The current model is geometric, not a semantic segmentation system. It may infer a roof or canopy as closer to a nadir camera because of learned shape, texture, boundaries, and context, but it is not guaranteed to obey “road = low, building = high.” Reliable class-specific height would need semantic fusion, remote-sensing training, and a ground model.

3 · Geospatial concepts without the jargon

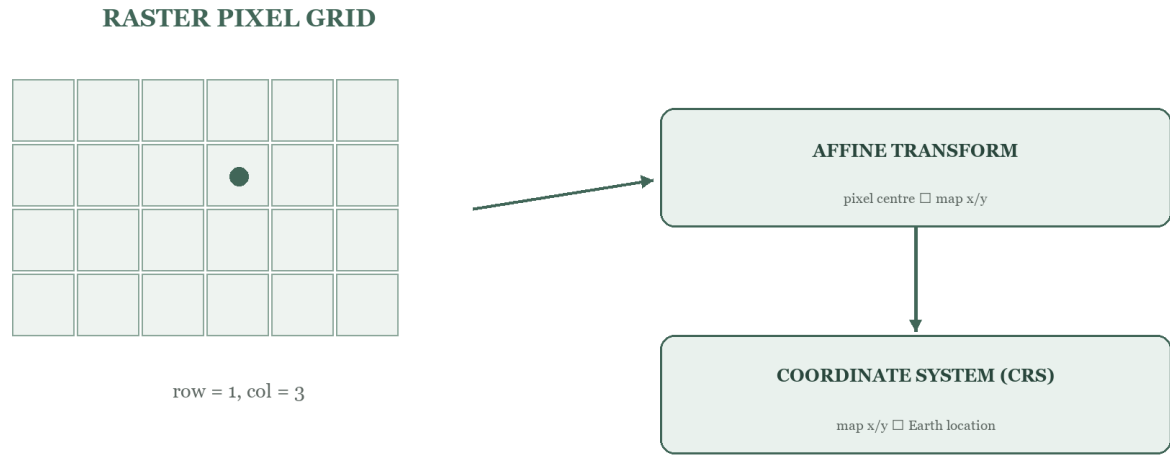


Figure 3. A pixel location becomes a map coordinate through an affine transform, then gains Earth meaning through the CRS.

Term	Beginner explanation	Why it matters here
CRS	The rulebook that gives coordinates a meaning on Earth.	Without it, x=500000 and y=3000000 are just numbers.
Geographic CRS	Coordinates are angular latitude/longitude, commonly degrees.	Degree spacing must not be treated directly as metres.
Projected CRS	A map projection expresses location on a flat grid, often in metres.	Useful for physical X/Y scale and slope.
Affine transform	Six coefficients mapping pixel column/row to map x/y.	Preserves the image footprint, origin, pixel size, and rotation.
Pixel resolution	Ground spacing represented by a pixel, when metadata is valid.	Controls horizontal scale in the viewer and raster alignment.
Bounding box	The left, bottom, right, and top map extent.	Checks whether reference rasters overlap.
NoData	A value or mask meaning “no valid observation here.”	Prevents borders and gaps from becoming terrain cliffs.
Vertical datum	The reference surface from which height is measured.	Different datums can add a systematic vertical offset.

GeoTIFF is more than a TIFF file

A GeoTIFF stores raster pixels plus geospatial metadata such as a CRS and affine transform. A .tif extension alone is not proof of correct georeferencing. The current Person 1 code transforms the image centre to longitude/latitude and checks whether it falls inside the declared CRS area of use. If the tag is inconsistent, it keeps the declared CRS for diagnosis but blocks metric calibration.

Why ordinary image resizing is not reprojection

A visual resize changes array dimensions. Reprojection asks which Earth location each destination pixel represents, then samples the source at that location. Person 3 uses rasterio.warp.reproject for SRTM/reference alignment because cv2.resize or a generic image resize does not understand CRS, bounds, rotation, or pixel origin.

4 • Intended architecture versus verified implementation

The intended conceptual flow

Image → preprocessing → relative depth → optional calibration → height surface → API → textured 3D viewer → website

What the repository actually does

The browser starts the work, so Person 5 is not merely a stage after Person 3. React sends a multipart upload to FastAPI. The backend creates a job and invokes Persons 1, 2, and optionally 3 in sequence. It then prepares or copies a bounded heightmap.json. React polls the job and lazily loads Person 6’s Three.js viewer on the results page.

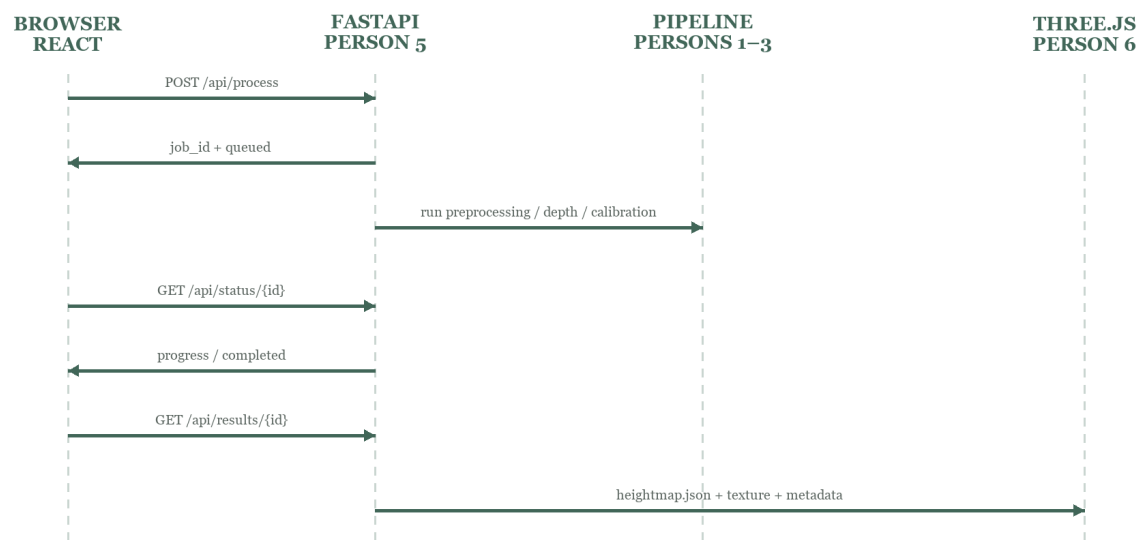


Figure 4. Technically accurate request and data sequence. The backend remains the orchestrator throughout processing.

Concern	Owner	Important boundary
Image integrity and metadata	Person 1	Does not estimate depth or create a DSM.
Relative geometry	Person 2	Does not invent metres or read CRS.
Absolute reference and DSM	Person 3	Requires valid georeferencing plus SRTM/GCP.
User interaction	Person 4	Does not perform heavy ML/GIS work in the browser.
Job/API integration	Person 5	Coordinates modules; does not own their algorithms.
3D visualization and probing	Person 6	Rendering quality does not validate height accuracy.

5 • Person 1 — Image loading, GeoTIFF metadata, and preprocessing

Role	What this module owns
Mission	Protect the image grid and geospatial meaning while creating a safe model input.
Input	PNG/JPG/JPEG/TIFF/GeoTIFF
Output	rgb_model.png, valid_mask.npy, metadata.json, preview.png; optional rgb_original.npy
Connection	Feeds RGB to Person 2 and grid/CRS/mask metadata to Person 3 and Person 5.

What this module does from scratch

1. Validate that the path exists and the extension is supported.
2. Read PNG/JPEG with Pillow or TIFF/GeoTIFF with Rasterio.
3. Convert band-first raster data into height × width × 3 RGB order.
4. Build a valid-pixel mask from NoData information and non-finite values.
5. Detect edge-connected near-black scanner/film borders when a TIFF declares no NoData value.
6. Validate that a claimed CRS and affine transform are geographically plausible.
7. Decode very large imagery directly to a bounded working grid whose long edge is at most 3072 pixels.
8. Normalize each display/model channel with valid-pixel 2nd and 98th percentiles, leaving the original numeric array unchanged.
9. Write stable files and metadata that downstream modules can trust.

Bands and array orientation

Rasterio reads bands × rows × columns. Neural-network image code normally expects rows × columns × channels. The module moves the band axis once. If a TIFF has one band, the band is repeated three times; if it has two, the second band is repeated as the third channel. This creates an RGB-shaped input, but it does not claim the data is true natural color.

```
Rasterio: (bands, rows, columns) → model image: (rows, columns, 3)
```

How a pixel retains its Earth location

The output metadata records the original and working dimensions plus scale factors. For a bounded grid, downstream code uses a correspondingly scaled affine transform rather than pretending the small grid has the original pixel spacing. Integer row/column coordinates refer to pixel centres for round-trip conversions.

Actual files

File	Purpose	Who uses it
depthwizard_person1/main.py	CLI orchestration and readable error handling	Backend adapter / direct users
src/image_loader.py	Input detection and Pillow path	Person 1
src/geotiff.py	Raster loading, masks, border detection, CRS validation	Person 1
src/preprocessing.py	Robust per-channel normalization	Person 1
src/coordinates.py	Pixel ↔ map and map ↔ lon/lat helpers	Persons 1 and 3 concepts
src/output.py	Writes the stable handoff package	Persons 2, 3, and 5

Input and output contract

Output	Grid / type	Meaning
rgb_model.png	Working $H \times W \times 3$, uint8	Aligned model texture/input; normalized only for inference/display
valid_mask.npy	Working $H \times W$, bool	True where pixels are valid scene content
metadata.json	JSON	Dimensions, CRS, transform, bounds, resolution, NoData, scale mapping, warnings
preview.png	≤ 1200 px preview	Human inspection only; never use for pixel-aligned depth
rgb_original.npy	Original grid and dtype	Optional large preservation copy; backend skips it to reduce duplication

Common failure cases

- CRS tag but impossible coordinates: mark non-georeferenced and warn; do not calibrate.
- Black border interpreted as deep terrain: mask only edge-connected near-black background, not all dark pixels.
- Huge compressed GeoTIFF expands in memory: decode to the bounded model grid and skip the duplicate original array in web jobs.
- Normalization makes original data unrecoverable: keep rgb_original.npy optional and record that rgb_model.png is a display/model transform.
- Depth returned flipped or transposed: keep row/column conventions explicit and test orientation.

Likely judge question	Strong, simple answer
Why use GeoTIFF?	It carries the CRS and pixel-to-map transform needed to place estimates on Earth and align external elevation.
What happens for PNG/JPG?	We still estimate relative structure, but no Earth coordinates or metres are invented.
What is an affine transform?	Six coefficients that map image column/row to map x/y, including scale, origin, and possible rotation.
Why preserve metadata?	If preprocessing loses the mapping, Person 3 cannot align SRTM or export a correctly located DSM.
Does every TIFF contain correct georeferencing?	No. We require a meaningful transform and CRS and validate their geographic plausibility.

Person 1 — 30-second explanation

I built the ingestion layer. It accepts ordinary images and GeoTIFFs, converts their bands into a consistent RGB grid, masks NoData and scanner borders, validates georeferencing, and creates a bounded normalized image for the depth model. I also preserve the dimensions, CRS, affine transform, bounds, resolution, and scale mapping in metadata so later calibration and 3D rendering do not lose the connection between a pixel and its Earth location.

6 • Person 2 — Monocular relative depth estimation

Role	What this module owns
Mission	Infer image-aligned relative geometry from one RGB view.
Input	rgb_model.png and optional valid_mask.npy
Output	relative_depth.npy, relative_depth_preview.png, depth_metadata.json, heightmap.json
Connection	Receives the prepared grid from Person 1 and supplies a unitless surface to Person 3 and Person 6.

What monocular depth means

Monocular means one camera view. A network learns statistical cues that often correlate with geometry: perspective convergence, relative size, texture density, occlusion boundaries, shadows, horizon position, and the typical shapes of objects. It predicts a value for each pixel, creating a depth map. Because one picture lacks triangulation, the result is not uniquely determined and must be treated as an informed estimate.

The model actually used

The current default is the pretrained depth-anything/Depth-Anything-V2-Base-hf checkpoint loaded through Hugging Face Transformers. The repository also exposes Depth Anything V2 Small/Large and MiDaS DPT Hybrid options. Base is the local RTX default because it preserves more detail than Small while remaining practical on the tested 8 GB GPU.

DIRECTION OF THE NUMBERS The current raw output is recorded as relative inverse depth: a larger value normally means closer to the camera. For a near-nadir aerial view, closer often corresponds to a higher surface—but this is not a semantic or metric guarantee.

Large-scene inference

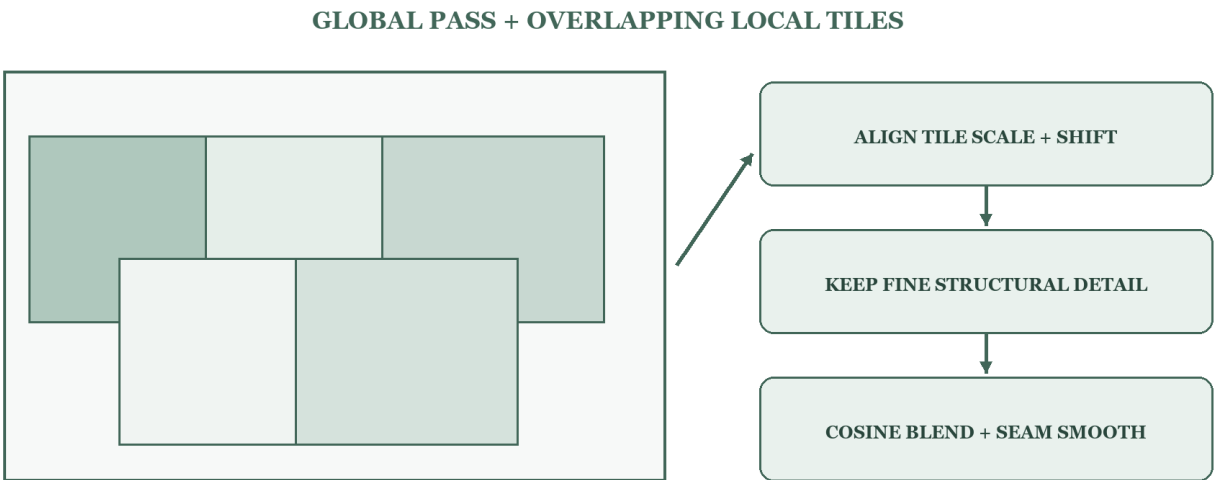


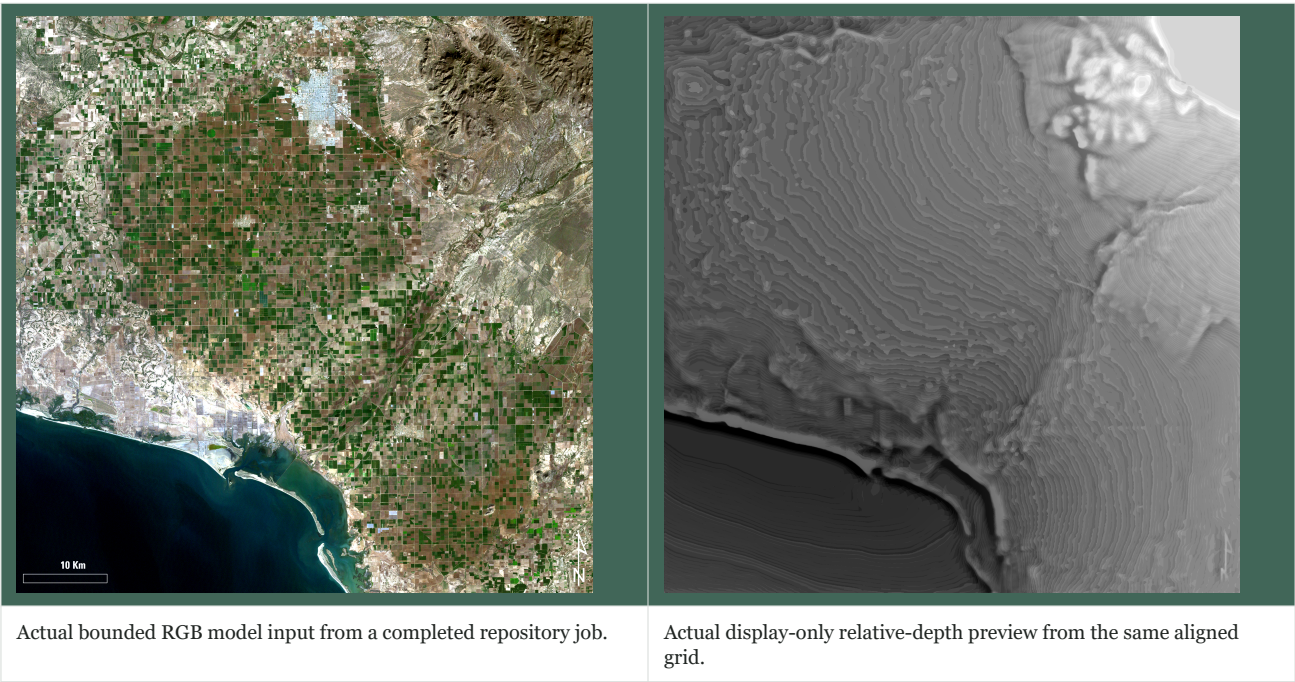
Figure 5. A scene-wide pass stabilizes the global trend; overlapping local passes recover higher-frequency structure.

1. Choose CUDA when torch.cuda.is_available(), otherwise CPU. CUDA inference uses float16 autocast and enables TF32 matrix multiplication where supported.
2. For an image no larger than 1024 pixels on either side, run a single prediction.
3. For a larger working image, run one scene-wide pass and overlapping 768-pixel tiles with 128-pixel overlap.

- 4. Linearly align each tile’s arbitrary scale and offset to the global prediction.
- 5. Keep the tile’s fine residual instead of allowing local low-frequency bias to create visible rectangles.
- 6. Blend tiles with cosine weights and smooth only a narrow stitched-detail residual at seams.
- 7. Restore the depth map to the Person 1 working grid and apply the valid mask as NaN.

Numerical output versus grayscale preview

relative_depth.npy is the scientific handoff and is not forced to 0–1. The PNG is for human viewing. Its contrast uses sampled 2nd/98th percentile limits, a modest histogram-equalization blend, and a locally detrended detail layer. Those operations improve visibility only; they do not change the NPY array used for calibration or point measurement.



Why Person 3 is still necessary

A relative prediction has no stable zero and no universal scale. It cannot state that a roof is 142.3 m above a datum. Person 3 pairs depth values with known reference heights and fits an approximate conversion before producing a metric surface.

Actual files

File	Responsibility
depthwizard_person2/config.py	Model choices and default checkpoint
src/depth_model.py	Device selection and pretrained model loading
src/inference.py	Single-pass/tiled prediction, alignment, detail merge, blending
src/postprocessing.py	Resize, finite-value handling, and external mask application
src/output.py	NPY, metadata, robust grayscale preview, and bounded heightmap JSON

Failure cases and responsible answers

- Nadir aerial imagery may lack the perspective cues common in ground-level datasets.
- Roofs and tree crowns can be confused; shadows may be interpreted as shape.
- Repeated textures, water, clouds, haze, and weak-texture surfaces can fail.

- A model can create plausible but false geometry; visual plausibility is not validation.
- Scene-wide scale can drift from image to image; calibration is required for metres.

Likely judge question	Strong, simple answer
How can AI estimate depth from one picture?	It learns statistical visual cues from many examples, but the answer remains ambiguous and approximate.
Is it measuring height?	No. The current backbone produces relative inverse-depth-like values, not elevation in metres.
Why Depth Anything V2 Base?	It gives stronger detail than Small and fits the tested 8 GB RTX workflow; Small remains available for CPU.
Does it know a road is ground?	No explicit semantic rule. It may infer geometry from appearance, but we do not guarantee class-specific height.
How do you reduce tile seams?	We align local tiles to a global pass, merge only fine residuals, cosine-blend overlaps, and lightly smooth the stitched residual.

Person 2 — 30-second explanation

I implemented monocular relative-depth inference using Depth Anything V2 Base. The model predicts an image-aligned unitless depth signal from one RGB view. For large aerial scenes, I combine a global pass with overlapping local tiles so the global trend stays consistent while roofs, roads, and canopy boundaries keep more detail. The numerical output remains raw relative depth; only the grayscale preview is contrast-enhanced. Person 3 then supplies the missing absolute scale.

7 · Person 3 — SRTM/GCP calibration and DSM generation

Role	What this module owns
Mission	Place relative depth on an approximate metric elevation scale and preserve geospatial alignment.
Input	Valid GeoTIFF metadata, relative_depth.npy, optional SRTM/elevation raster, optional GCP CSV/JSON, optional reference DSM
Output	absolute_dsm.tif/.npy, preview_dsm.png, metadata.json, metrics.json
Connection	Consumes Persons 1–2 outputs; Person 5 serves products to Persons 4 and 6.

Why SRTM is useful—but not enough

The Shuttle Radar Topography Mission (SRTM) is a radar-derived elevation product, not a model we train. Common global 1-arc-second data is roughly 30 m spacing near the equator and is typically supplied in geographic WGS84 with metre elevations referenced to EGM96 for the cited product. It is valuable as a broad absolute anchor, but one coarse cell cannot resolve every road, roof, or tree in a high-resolution optical image.

Resampling SRTM to a smaller optical pixel size does not create new detail. It only puts the available coarse information onto the target grid. DepthWizard therefore uses SRTM mainly for low-frequency elevation and uses the calibrated monocular signal for estimated high-frequency variation.

What a GCP contributes

A Ground Control Point (GCP) is a location where we know an elevation and either the source-image row/column or WGS84 longitude/latitude. At least two distinct valid points are needed to fit both scale and offset. More well-distributed, accurate points improve stability; clustered or incorrect points can bias the entire scene.

```
Pixel GCP: name,row,col,elevation_m
WGS84 GCP: name,longitude,latitude,elevation_m
```

CRS-aware alignment

1. Read the RGB GeoTIFF target grid: CRS, affine transform, width, height, bounds, resolution, and NoData.
2. If Person 1 used a bounded depth grid, derive a scaled affine transform that preserves the original outer footprint.
3. Reproject SRTM and any independent reference DSM into the target CRS, transform, width, and height with Rasterio.
4. Check exact shape, CRS, and transform compatibility before comparing or fusing arrays.
5. Transform longitude/latitude GCPs into the target CRS, then map coordinates to row/column.

Calibration candidates

Candidate	Equation	When it may help	Caution
Linear	$H = aD + b$	Raw depth behaves approximately linearly with height	Global assumption can miss local perspective effects
Inverse	$H = a(1/D) + b$	Model output behaves more like inverse depth	Zero/invalid D must be excluded
Robust linear	Huber fit	A few reference pairs are outliers	Robust is not immune to bad or clustered controls

With 4–50 samples the code computes leave-one-out diagnostics; larger SRTM sample sets use a deterministic holdout. Automatic selection prefers the candidate with the best available validation RMSE. These diagnostics measure fit to the supplied calibration pairs, not independent scene accuracy.

Fusion

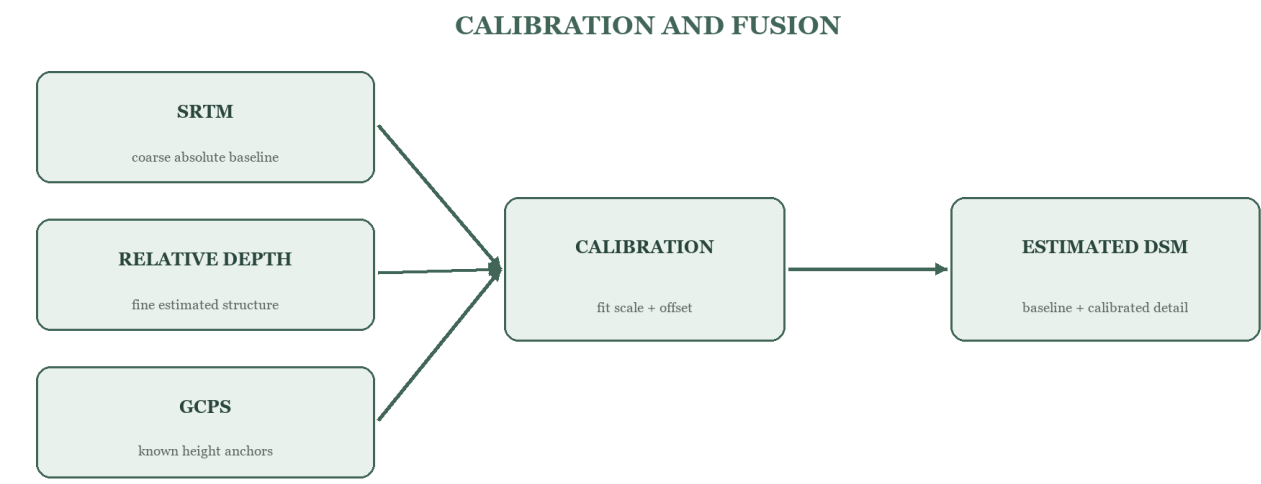


Figure 6. Coarse absolute reference and fine estimated structure are combined only after calibration.

```
depth_smooth = Gaussian_smooth(calibrated_depth)
depth_detail = calibrated_depth - depth_smooth
final_dsm = smoothed_aligned_srtm + alpha * depth_detail
```

The alpha parameter controls detail strength and defaults to 1.0. If only GCPs are supplied, the code exports the GCP-calibrated depth estimate without an SRTM baseline. Raw relative depth is never simply added to SRTM.

SRTM-only calibration

When no GCPs are provided, the code smooths the relative depth to approximately the effective SRTM scale, pairs those coarse values with aligned SRTM heights, and fits a global mapping. This is a practical heuristic, not proof that the fine monocular surface is correct.

Output and validation

Output	Meaning
absolute_dsm.tif	Float32 geospatial raster, metre values, working CRS/transform/footprint, NoData -9999
absolute_dsm.npy	Numeric array used by the heightmap converter
preview_dsm.png	Colored human preview with a metre-labelled colorbar
metadata.json	Calibration source, candidates, coefficients, grid, ranges, fusion, limitations
metrics.json	MAE, RMSE, Pearson r, and bias when an independent reference DSM is supplied

Metric	What it tells us	What it does not prove
MAE	Average absolute vertical error	Whether a few extreme errors exist
RMSE	Penalizes large errors more strongly	Why the errors occurred
Pearson r	Similarity of spatial variation	Correct vertical offset or scale
Bias	Average prediction minus reference	Local accuracy

Vertical datum caution

Two datasets can share horizontal coordinates and still disagree vertically because their zero surfaces differ. Ellipsoidal height and orthometric height are not automatically interchangeable. The prototype records assumptions but does not implement a full geoid/datum transformation pipeline; mismatched references may create systematic offsets.

What happens when calibration is absent

The web pipeline does not fail ordinary images. Person 5 copies Person 2’s relative NPY, preview, and heightmap into the result contract and marks `is_absolute_elevation=false`, `elevation_units=relative`, and `calibration_source=Absolute` elevation unavailable. Person 3’s direct CLI correctly requires SRTM or GCPs because its sole purpose is absolute calibration.

Likely judge question	Strong, simple answer
Why not use SRTM alone?	It anchors broad elevation but is too coarse to describe individual small structures in high-resolution imagery.
How is calibration performed?	We align reference and depth grids, sample paired values, fit scale and offset with candidate regressions, then fuse calibrated detail with a coarse baseline.
Is the DSM accurate?	It is an estimated/fused DSM. Accuracy is scene- and reference-dependent and must be reported against independent LiDAR or reference DSM.
Can this replace LiDAR?	No. LiDAR directly measures geometry; our single-view result is an accessible prototype with much larger uncertainty.
What if SRTM and depth disagree?	We preserve SRTM as the low-frequency anchor and limit monocular influence to calibrated detail; disagreement is a validation warning, not something to hide.

Person 3 — 30-second explanation

I convert the unitless depth map into an estimated geospatial height surface. I align local SRTM or known GCP elevations to the image grid, fit the depth-to-height scale and offset using linear, inverse, and robust candidates, and combine SRTM’s coarse absolute baseline with the model’s estimated fine detail. I export a GeoTIFF DSM and record calibration, validation, and datum limitations. Without trustworthy reference data, the system stays honestly relative.

8 · Person 4 — Frontend website and user experience

Role	What this module owns
Mission	Make the scientific workflow understandable, controllable, and honest for a user.
Input	User-selected image plus optional SRTM/GCP files; API status/results JSON
Output	Upload request, progress UI, previews, metadata, downloads, and embedded 3D viewer
Connection	Calls Person 5’s API and hosts Person 6’s viewer component on the results page.

Actual stack

The frontend is a React application built with Vite. React Router provides the Home, Analyze, and Results pages. The Three.js viewer is lazy-loaded only on the results page so the main application does not pay the large renderer bundle cost up front.

User flow

1. Choose PNG, JPG/JPEG, TIF, or TIFF and see a local preview when the browser supports it.
2. Optionally open Advanced calibration and attach a GCP CSV/JSON and/or SRTM TIFF/HGT. The UI rejects calibration files for a non-TIFF source.
3. Submit a multipart POST request to /api/process.
4. Show processing status while polling /api/status/{job_id}; retry transient polling failures with increasing delay.
5. Navigate to /results/{job_id} when the job completes.
6. Display relative-depth and DSM previews, metadata, warnings, downloadable files, and the 3D viewer.

Communicating uncertainty

The interface changes units and explanatory text based on is_absolute_elevation. Calibrated results show metres and a validation caution. Relative results show rel and explicitly explain that the values are not measured metres or guaranteed semantic class heights. This is part of the technical design, not merely copywriting.

Actual files

File	Responsibility
src/pages/HomePage.jsx	Project story and five-step overview
src/pages/AnalyzePage.jsx	Upload validation, calibration inputs, job creation, polling
src/pages/ResultsPage.jsx	Result discovery, previews, downloads, lazy viewer
src/services/realApi.js	Multipart upload and GET calls to FastAPI
src/components/MetadataPanel.jsx	Mode-aware units, ranges, model/calibration facts, warnings
src/components/TerrainViewer.jsx	Lifecycle wrapper around Person 6’s createTerrainViewer
src/styles/global.css	Responsive visual system and states

Why heavy computation stays out of the browser

The browser is excellent for interaction and WebGL rendering but not ideal for running a large PyTorch geospatial pipeline on arbitrary client devices. The backend centralizes Python, Rasterio, model weights, CUDA selection, files, and errors. The browser receives bounded previews and heightmap JSON, while full NPY/GeoTIFF products remain downloadable.

Likely judge question	Strong, simple answer
What does the frontend do?	It validates user choices, uploads files, shows progress and errors, presents results, and embeds the interactive viewer.
How does it communicate with the backend?	Through HTTP: one multipart POST to create a job and GET requests for status, results, and files.
How do you show uncertainty?	We change units and warnings by mode: metres only for reference-calibrated runs, rel otherwise.
What happens if the backend is down?	The API wrapper reports a clear reachability error instead of silently failing.
Why React/Vite?	React organizes stateful upload/result components; Vite provides a fast development server and compact production build.

Person 4 — 30-second explanation

I built the React/Vite user experience. Users upload an image, optionally attach SRTM or GCP calibration, see progress and clear errors, and then explore previews, metadata, downloads, and the 3D viewer. The frontend talks to FastAPI through a small HTTP service layer. Most importantly, it distinguishes relative results from calibrated metre results so the interface does not overstate what the model knows.

9 • Person 5 — Backend and API integration

Role	What this module owns
Mission	Turn six separate modules into one reliable request-to-result workflow.
Input	Multipart upload: image plus optional srtm and gcp
Output	Job ID/status JSON, result metadata and safe file URLs
Connection	Invokes Persons 1–3, prepares Person 6 data, and serves Person 4.

Backend and API basics

A backend is the server-side part of the application. An API is a documented set of requests and responses. HTTP POST usually sends or creates data; HTTP GET retrieves data. JSON is the small text format used for job state and result metadata. Large binary products are served as files rather than inserted into JSON.

Actual stack and endpoints

The implementation uses FastAPI and Uvicorn. CORS allows common localhost frontend ports. FastAPI BackgroundTasks starts processing after returning the job ID, so a large model run does not keep the upload request open.

Method	Route	Purpose
GET	/health	Confirm that the backend is alive
POST	/api/process	Validate/save image and optional SRTM/GCP; queue a job
GET	/api/status/{job_id}	Return public status, stage, progress, or safe failure message
GET	/api/results/{job_id}	Return metadata and URLs after completion
GET	/api/files/{job_id}/{filename}	Download/display an allowed generated file

Stage ordering and progress

15% preprocessing → 45% depth estimation → 75% calibration/fallback → 100% completed

pipeline_runner.py uses sys.executable and argument lists, never shell=True. Every stage has a 30-minute default timeout, required-output checks, captured stdout/stderr, and a readable label. A failure stops later stages and writes private diagnostics to the job's status.json while exposing only a safe browser message.

Data and file safety

- Random job IDs match job_[0-9a-f]{12}; each job has isolated input, person1, person2, person3, and results directories.
- The default image limit is 500 MB; uploads are streamed in 1 MB chunks instead of read entirely into HTTP memory.
- Extensions and file signatures are validated; empty or mismatched files are rejected.
- HGT files keep names such as N28E077.hgt because GDAL derives their location from the tile name.
- Plain-filename checks and fixed search directories prevent traversal through the file endpoint.
- Large arrays are hard-linked when possible, with copy fallback, to avoid unnecessary multi-gigabyte duplication.

Relative versus calibrated branch

If no calibration file or configured reference exists, the backend does not call Person 3’s absolute CLI. It exposes Person 2’s aligned relative surface and writes a `calibration_report.json` that explicitly marks absolute elevation unavailable. If SRTM or GCP is supplied, it first confirms Person 1 accepted the georeferencing, then invokes Person 3 and converts `absolute_dsm.npy` to a bounded heightmap.json.

Result contract

Result field	Consumer	Meaning
depth_preview_url	Person 4	Human-readable relative-depth PNG
dsm_preview_url	Person 4	Absolute preview or relative fallback preview
dsm_download_url	User	Absolute DSM/relative NPY download
heightmap_url	Person 6	Bounded JSON grid—not a raw NPY/TIFF
texture_url	Person 6	Aligned bounded RGB PNG
metadata_url	Persons 4 and 6	Grid, mode, calibration, units, transform, limits

Prototype scaling boundary

FastAPI BackgroundTasks and local job folders are suitable for a hackathon machine. A production city-scale service should use object storage, a durable queue, dedicated GPU workers, retry/idempotency policies, resource quotas, authentication, observability, and tiled geospatial output rather than one large synchronous local filesystem workflow.

Likely judge question	Strong, simple answer
What is an API?	A defined set of HTTP requests and responses that lets the frontend use backend processing without knowing its internal code.
Does the backend implement the model?	It orchestrates the owner modules through subprocess adapters; Person 2 owns inference and Person 3 owns calibration.
What happens when a stage fails?	Later stages stop, the job becomes failed, private logs remain on disk, and the browser receives a safe actionable message.
Why background jobs?	The upload returns quickly with a job ID, and the frontend can poll during long model inference.
How do modules exchange data?	Through stable files—PNG, NPY, GeoTIFF, and JSON—inside an isolated job directory.

Person 5 — 30-second explanation

I built the FastAPI integration layer that turns separate modules into one application. The backend validates and streams uploads, creates an isolated job, runs preprocessing, depth, and optional calibration in order, prepares the browser heightmap, and exposes status, metadata, previews, and downloads through safe endpoints. If a stage fails, the pipeline stops cleanly and returns a useful error. If calibration is absent, it deliberately returns a relative result instead of inventing metres.

10 • Person 6 — 3D terrain rendering and flythrough

Role	What this module owns
Mission	Convert an aligned height grid and RGB image into an explorable WebGL terrain without changing reported measurements.
Input	heightmap.json, rgb_model.png, metadata.json
Output	Interactive Three.js scene, point elevation/slope HUD, overview/free flight/auto path
Connection	Embedded by Person 4; data and URLs supplied by Person 5.

Height map to mesh

HEIGHT SAMPLES □ VERTICES □ TRIANGLES

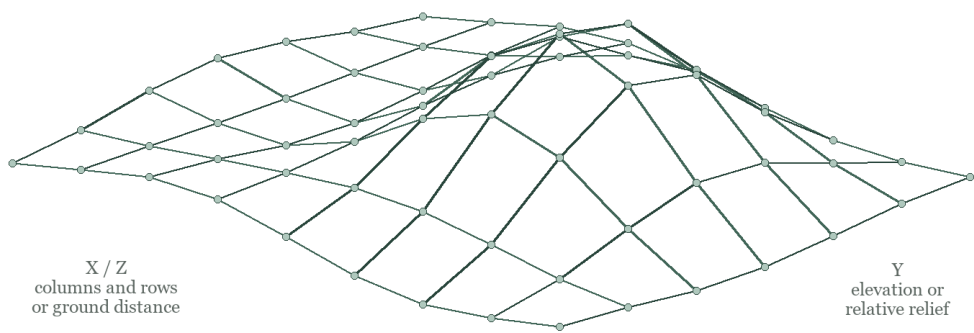


Figure 7. Every sampled height becomes a vertex. Two triangles per valid grid cell form a continuous surface.

A mesh is a set of vertices connected into faces, usually triangles. For each height sample, X comes from the column, Z from the row, and Y from the height above a baseline. Neighboring vertices are connected into triangles because graphics hardware is optimized to render triangles and any planar surface patch can be represented by them.

```
grid sample (row, col, height) → vertex (X = col, Y = height, Z = row)
```

Physical versus relative scaling

For a calibrated projected raster, horizontal spacing comes from pixel resolution and the CRS unit conversion; vertical values are metres. For EPSG:4326/CRS84, the code approximately converts longitude/latitude degree spacing to local metres at the scene centre. For a relative result, the unitless range is clipped to its 1st/99th percentiles for display and scaled to a conservative relief equal to about 3% of the scene width.

Rendering quality features in the current code

- At most 512 terrain samples on the long edge (about 262,000 vertices maximum) with bilinear reduction to reduce aliasing.
- Triangles touching invalid source pixels are removed, preventing masked scan borders from becoming walls.
- Two conservative display-only smoothing passes for relative geometry; raw values remain in elevationHeights for the HUD.

- RGB texture with mipmapping, linear filtering, and up to 16× anisotropy depending on GPU capability.
- WebGL antialiasing, sRGB output, ACES filmic tone mapping, soft PCF shadows, and a 2048×2048 shadow map.
- Device pixel ratio capped at 2× and automatic resize handling for stable performance.
- GPU badge reads the WebGL renderer string and warns when an integrated or software renderer is selected.

Texture and UV mapping

UV coordinates tell the graphics engine which RGB pixel belongs on each vertex. The convention is deterministic: height[row, col] corresponds to image pixel (col, row), and the top raster row remains the top texture row. Texture alignment requires the RGB and height surface to describe the same crop and orientation.

Controls that actually exist

Mode/control	Behavior
Overview	Orbit with drag, pan with right-drag, zoom with wheel
Fly	Click viewport for mouse look; W/S forward/back; A/D strafe; Q or Space up; E or Shift down
Auto	24-second closed Catmull–Rom path; pause/resume/reset
Texture / Elevation / Wireframe	Switch visual material without changing data
Relief	0.2×–2× for relative mode; 0.2×–5× for metric mode
Speed / Look	Adjust movement speed and pointer sensitivity
Centre reticle	Reports aimed value, slope, source pixel, calibration source, and map coordinate when available

How point values are calculated

A ray from the camera through the centre reticle intersects the mesh. The viewer maps that point back to grid coordinates and bilinearly samples the unmodified numeric height field. For metric data, local rise over horizontal metre spacing gives a physical slope. For relative data, the HUD labels the slope visual because the vertical scale is arbitrary.

GPU use

There are two separate GPU workloads. PyTorch/CUDA runs model inference on the backend. WebGL renders the mesh in the browser. The renderer requests high-performance mode, but that is only a browser hint; the included launcher starts an isolated Chromium-family session with a high-performance GPU preference, and the badge confirms whether NVIDIA/RTX is actually selected.

Likely judge question	Strong, simple answer
How do you make 3D from 2D?	Each height sample becomes a vertex, adjacent vertices become triangles, and the original RGB image is mapped with UV coordinates.
Why downsample?	A browser does not need one vertex for every source pixel; bounded bilinear sampling preserves shape while keeping interaction smooth.
Why do A/D move sideways?	They add/subtract the camera’s right vector—translation—while mouse movement changes yaw/pitch—rotation.
Are HUD heights real?	Only calibrated jobs report estimated metres; uncalibrated jobs report rel and visual slope.
Does smoothing alter measurements?	No. Smoothing changes displayed vertex positions only; the HUD samples the original height field.

Person 6 — 30-second explanation

I turn the heightmap and aligned RGB image into an interactive Three.js terrain. Height samples become mesh vertices, valid neighbours become triangles, and UV coordinates place the original image on the surface. I added overview, free flight, auto flythrough, texture/elevation/wireframe modes, relief controls, GPU diagnostics, and a centre probe for height, slope, pixel, and map coordinates. Relative smoothing improves appearance without changing the values we report.

11 • Complete system integration

End-to-end data flow

Step	Producer ☐ consumer	Artifact / request	Invariant
1	Person 4 ☐ Person 5	multipart image; optional srtm/gcp	Supported type and valid signature
2	Person 5 ☐ Person 1	source path, output directory, 3072 max grid	No shell interpolation
3	Person 1 ☐ Person 2	rgb_model.png + valid_mask.npy	Same working H × W and orientation
4	Person 1/2 ☐ Person 3	metadata.json + relative_depth.npy + references	Valid CRS and aligned/derived grid
5a	Person 3 ☐ Person 5	absolute_dsm.npy/.tif + metadata	Metric only after reference calibration
5b	Person 2 ☐ Person 5	relative depth/heightmap fallback	Units remain relative
6	Person 5 ☐ Person 6	heightmap.json + texture + metadata URL	Bounded JSON, not raw NPY
7	Person 5 ☐ Person 4	results JSON + file URLs	Mode and warnings preserved

The non-georeferenced path

PNG/JPG → bounded RGB → relative inverse depth → relative heightmap.json → rel-labelled 3D terrain

No CRS, transform, map coordinate, or metre elevation is invented. The image can still produce a visually useful relative scene, but model ambiguity remains.

The calibrated GeoTIFF path

Valid GeoTIFF + SRTM/GCP → aligned references → calibrated detail + coarse baseline → estimated absolute DSM → metre-labelled 3D terrain

The path is only taken if Person 1 accepts the georeferencing and the user/configuration supplies at least one calibration source.

Why dimensions are a team-wide contract

If a teammate flips, crops, transposes, or independently resizes one array without recording the mapping, a roof in the RGB image may receive a road’s height. The repository therefore preserves working-grid orientation, stores model-to-original scale, derives a CRS-preserving target grid for calibration, and uses explicit source/target dimensions in viewer metadata.

What is downloaded versus displayed

Need	Format	Reason
ML/GIS numeric work	NPY or GeoTIFF	Retains float values and, for GeoTIFF, geospatial metadata
Human 2D inspection	PNG preview	Small and universally displayable; not a numeric product
Browser terrain	heightmap.json	Easy fetch/parse with bounded sample count and validity mask
Texture	rgb_model.png	Browser-compatible and aligned to the working grid
Provenance	metadata.json / metrics.json	Records model, mode, calibration, grid, ranges, warnings, validation

12 · Validation, evaluation, and what good evidence looks like

Current automated verification

Suite	Passed	What it checks
Person 1	4	PNG/TIFF/GeoTIFF loading, round-trip, border mask, invalid CRS, CLI outputs
Person 2	5	Shape/orientation, finite output handling, mask behavior, tiled seam/detail behavior
Person 3	7	Alignment, bounded grids, GCPs, SRTM fusion, metrics, invalid CRS rejection
Person 5	8	Upload/API contracts, safety, stage results and failure behavior
Person 6	8	Camera axes, JSON loader, physical extent, nodata triangles, sampling, display-only smoothing
Frontend build	Pass	React/Vite production compilation; 48 modules transformed

VERIFIED STATE On 5 September 2026, all 32 automated tests passed and the production Vite build completed. Rasterio/Starlette deprecation warnings were present, but there were no test failures.

Accuracy study the team should run

1. Select independent reference DSM/LiDAR scenes that were not used as GCPs or calibration inputs.
2. Stratify scenes: dense urban, sparse built-up, hilly, forested, barren, and water/cloud edge cases.
3. Align prediction and reference to the same horizontal CRS, footprint, resolution, mask, and vertical datum.
4. Report MAE, RMSE, Pearson r, bias, valid-pixel coverage, and runtime; do not report only one attractive example.
5. Compare baselines: SRTM alone, monocular relative structure, GCP-only calibration, and fused SRTM + GCP.
6. Perform ablations: Small vs Base model, single pass vs tiled, alpha values, number/distribution of GCPs.
7. Show error maps and failure examples alongside best cases.

How to validate the 3D experience

- Texture/height registration: identifiable roof corners and roads should align with relief boundaries.
- Navigation: overview, free flight, rise/fall, pointer lock, auto path, pause/resume/reset.
- Performance: interactive frame rate on target hardware and bounded memory for large scenes.
- Measurement integrity: point HUD equals the unmodified height field at known test pixels.
- Mode honesty: no metres, map coordinates, or metric slopes in the uncalibrated path.

13 • Important limitations and assumptions

Limitations are not an admission that the project failed. They define where the result is useful, what evidence is still needed, and how a responsible team would improve it.

Limitation	Effect	Responsible mitigation / future work
Monocular scale ambiguity	Relative values vary by scene and lack metres	Reference calibration; never label raw output as metric
Plausible hallucination	The model can invent incorrect geometry	Independent reference validation and uncertainty maps
Hidden surfaces	One image cannot reconstruct unseen sides	Multi-view imagery or LiDAR for complete geometry
Canopy/building confusion	Top surfaces and ground are not separated reliably	Remote-sensing fine-tuning + semantic classes + ground DTM
SRTM is coarse	Individual roofs/roads are unresolved	Use it as broad baseline, not detailed truth
SRTM surface bias	Radar response may differ from bare earth or optical top surface	Use product-specific quality layers and better local DEM
CRS mismatch	Reference pixels align to the wrong location	Validate CRS/transform/bounds and reproject geospatially
Vertical datum mismatch	Systematic height offset	Transform datums/geoid models before evaluation
GCP quality/distribution	Bad anchors distort scale and offset	Surveyed points, spatial coverage, robust fitting, holdout tests
No position in PNG/JPG	No map coordinates or absolute elevation	Remain relative unless external georeferencing is explicitly supplied
Texture can look convincing	Users may trust visually attractive but wrong geometry	Keep provenance, mode, warnings, and point evidence visible
Large scenes	High RAM, GPU memory, disk, and latency	Bounded working grid, tiles, browser downsampling; production tiling/queue
Prototype deployment	Single local process and local job storage	Durable workers, object storage, monitoring, authentication

DO NOT CLAIM “We recover exact building height from any single satellite image,” “SRTM is our trained AI,” “every GeoTIFF is valid,” “the 3D model proves accuracy,” or “this replaces LiDAR/photogrammetry.”

What the team can confidently claim

- A complete local pipeline from upload to relative or reference-calibrated height surface and interactive 3D visualization.
- Repository-tested handling of large imagery, masks, geospatial metadata, tiled inference, optional SRTM/GCP calibration, API jobs, and 3D navigation.
- Explicit separation of numerical data from display-only contrast, smoothing, and relief exaggeration.
- A practical platform for further remote-sensing fine-tuning and validation—not a finished measurement instrument.

14 • Demo runbook and troubleshooting

Fastest reliable startup on Windows

```
cd "C:\Users\preet\OneDrive\Documents\ChatGPT\Hackathon 2026 Depth wizard"
.\start_depthwizard.cmd
```

The .cmd launcher works even when PowerShell execution policy blocks .ps1 files. It starts separate backend and frontend terminals, waits for both HTTP services, and opens the high-performance GPU viewer. Manual .cmd launchers also exist: start_backend.cmd, start_frontend.cmd, and open_gpu_viewer.cmd.

If using PowerShell scripts

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\start_backend.ps1
# new terminal at the same repository root
.\start_frontend.ps1
```

Pre-demo checklist

- Run test_all.ps1 (or the .cmd launcher’s documented checks) before the event; keep one known-good sample ready.
- Verify <http://127.0.0.1:8000/health> and <http://127.0.0.1:5173> before uploading.
- For a metric demo, use a genuinely valid georeferenced GeoTIFF and an overlapping SRTM/GCP file.
- Confirm backend metadata says ML compute: CUDA and the viewer GPU badge says NVIDIA/RTX rather than Intel/software.
- Explain the mode before showing the terrain: relative (rel) or estimated absolute (m).
- Keep a short screen recording as a fallback, but demonstrate the live point HUD if the machine is stable.

Common startup problems

Symptom	Likely cause	Fix
.ps1 cannot be loaded	Execution policy	Use start_depthwizard.cmd, or set process-scoped RemoteSigned
node is not recognized	Node not on PATH	Use the root launcher, which tries pnpm/npm and the bundled runtime
Site cannot be reached	Frontend exited or port 5173 not ready	Read its terminal; verify the process stayed running
Backend unreachable	Uvicorn failed or wrong directory/environment	Open /health and inspect the backend terminal
Absolute elevation unavailable	No valid SRTM/GCP or georeferencing rejected	Provide compatible reference files and inspect the warning
Integrated GPU badge	Browser ignored high-performance hint	Use open_gpu_viewer.cmd/ps1 and re-check badge
3D cliffs at borders	Invalid mask missing or old job output	Reprocess with current Person 1/6 code
Texture does not align	Grid/crop/orientation mismatch	Trace Person 1 metadata and width/height through every handoff

15 · Possible judge questions and suggested answers

Use these as talking points, not a script to memorize word for word. The strongest answer states what works, what evidence supports it, and what remains uncertain.

1. What problem are you solving?

We convert one optical image into an approximate relative or calibrated height surface and make it immediately explorable in 3D. The calibrated path uses valid georeferencing plus SRTM and/or GCPs.

2. Why use a single view instead of stereo?

Single images are far more available and operationally simple. We accept the ambiguity and treat monocular output as a prior, not as direct survey measurement.

3. How can one image contain depth information?

Perspective, occlusion, texture, size, shadows, and scene context contain statistical cues. A pretrained network learns those cues, but multiple 3D scenes can still explain the same image.

4. What model do you use?

The default is depth-anything/Depth-Anything-V2-Base-hf through Transformers. Small, Large, and MiDaS DPT Hybrid are configurable alternatives.

5. Does that model output metres?

No. Our selected checkpoint produces scene-dependent relative inverse-depth-like values. We add approximate metric scale only through external reference calibration.

6. What does larger depth mean?

For the current raw representation, larger generally means closer to the camera. The direction is stored in `depth_metadata.json`.

7. Does the model know buildings are high and roads are low?

Not as an explicit semantic rule. It infers geometric appearance, so class-specific behavior can fail. Semantic fusion is future work.

8. Why Depth Anything V2 Base?

It offers stronger detail than Small while fitting the tested 8 GB RTX workflow. Base is a practical accuracy/performance compromise for the hackathon.

9. Why tiled inference?

Shrinking a large aerial image to one small pass destroys roofs and boundaries. We combine a global prediction with overlapping local tiles to retain detail and scene consistency.

10. How do you prevent tile seams?

Local tiles are scale/shift aligned to a global pass, reduced to fine residual detail, cosine blended across 128-pixel overlaps, and lightly seam-smoothed.

11. What is SRTM?

A radar-derived elevation dataset from the Shuttle Radar Topography Mission. It is reference data, not an AI model we train.

12. What is SRTM's spatial resolution?

For the global 1-arc-second product, sample spacing is about 30 m near the equator. Exact spacing varies with latitude and product.

13. Why not just use SRTM?

It gives broad absolute elevation but cannot resolve most individual buildings or narrow roads in a high-resolution optical scene.

14. What is a GCP?

A known elevation anchor associated with a source pixel or geographic coordinate. We use valid points to estimate scale and offset.

15. How is calibration performed?

We form reference height/depth pairs and compare linear, inverse, and Huber robust-linear mappings. Auto mode selects using cross-validated or holdout RMSE when available.

16. What is your fusion equation?

We separate calibrated depth into coarse and fine components, then add alpha times its fine detail to a smoothed aligned SRTM baseline.

17. What is a DEM?

A general elevation grid. Depending on the product, it may represent terrain or another surface; we always check the product definition.

18. What is a DSM?

A digital surface model represents the upper sensed surface, which may include roofs and vegetation. Our output is an estimated/fused DSM.

19. What is a DTM or nDSM?

A DTM is intended as bare earth. nDSM is DSM minus a reliable ground DTM, giving height above ground. Our prototype does not promise a reliable ground DTM.

20. What is a GeoTIFF?

A TIFF raster with geospatial metadata such as a CRS and affine transform, allowing pixels to be placed on Earth.

21. What is a CRS?

A coordinate reference system defines what map coordinate numbers mean and how they relate to Earth.

22. What is the affine transform?

Six coefficients mapping pixel column and row to map x and y, including origin, resolution, and possible rotation.

23. Why is ordinary resize not enough for SRTM alignment?

Resize knows only array dimensions. Reprojection uses both rasters' CRS, transforms, and footprints so samples correspond to the same Earth locations.

24. What is a vertical datum?

The zero reference for elevation. If SRTM, GCPs, and validation data use different vertical datums, their heights may differ by a systematic offset.

25. Is your DSM accurate?

It is an estimate whose accuracy depends on scene, model, references, alignment, and datum. We only quantify accuracy against an independent reference DSM/LiDAR.

26. How would you validate it?

Use held-out reference DSM/LiDAR, align grids and datums, mask invalid overlap, and report MAE, RMSE, correlation, bias, and error maps across scene types.

27. Can this replace LiDAR or photogrammetry?

No. Those methods directly measure geometry with stronger constraints. DepthWizard is a fast visualization and research prototype for situations with limited inputs.

28. What happens for PNG/JPG?

The pipeline still produces aligned relative depth and 3D relief, but units remain rel and no Earth coordinates or metre elevation are claimed.

29. What if a GeoTIFF has bad metadata?

Person 1 checks that the CRS and affine transform are meaningful and geographically plausible. Bad georeferencing blocks absolute calibration.

30. How does the frontend communicate with the backend?

React sends a multipart POST to create a job, polls GET status, then fetches GET results and file URLs.

31. Why FastAPI?

It provides typed HTTP routes, file uploads, automatic API docs, background tasks, and clear Python integration with the ML/GIS modules.

32. What happens if a stage fails?

The backend stops the pipeline, records the failed stage and private logs, marks the job failed, and returns a safe actionable message.

33. How do you make the 3D mesh?

The bounded height grid becomes vertices; valid neighboring samples become triangles; UV coordinates map the aligned RGB image onto the surface.

34. Why triangles?

GPUs render triangles efficiently, and a pair of triangles represents each grid cell without ambiguity.

35. Why downsample the terrain?

One vertex per large source pixel would waste browser memory and frame time. A 512-sample long edge preserves overview shape while keeping interaction smooth.

36. Does vertical exaggeration change the data?

No. It scales displayed vertex Y positions. The point HUD reads the original numeric field; relative slope is labelled visual.

37. Does the project use the GPU?

Yes: PyTorch CUDA accelerates depth inference when available, and WebGL uses the browser-selected GPU for rendering. They are separate workloads.

38. How much GPU memory does it use?

We tested the Base model on an 8 GB RTX 5060. Exact peak depends on model, tile size, and image; bounded 3072-pixel preprocessing and 768-pixel tiles control memory. We should profile before claiming a universal number.

39. What is the innovation?

The contribution is the transparent integration: large-image relative inference, optional geospatial calibration, honest dual-mode outputs, stable API contracts, and real-time textured analysis in one local workflow.

40. What would you improve next?

Fine-tune on aerial height datasets, add semantic/ground separation and uncertainty, automate high-quality DEM retrieval, reconcile vertical datums, validate on LiDAR benchmarks, and deploy durable GPU workers.

Cheat sheet • Person 1 — Image loading, GeoTIFF metadata, and preprocessing

Prompt	My answer
My module	Prepare a clean, aligned RGB grid while preserving or rejecting geospatial meaning correctly.
My input	PNG/JPG/JPEG/TIFF/GeoTIFF.
What I do	Validate format; read bands; build masks; detect scan borders; validate CRS/transform; bounded decode; percentile-normalize; record grid mapping.
My output	rgb_model.png, valid_mask.npy, metadata.json, preview.png; optional rgb_original.npy.
Why it matters	Every later module fails if orientation, validity, or pixel-to-Earth mapping is wrong.
Main technologies	Python, Rasterio/GDAL, Pillow, NumPy, PyProj, Affine.
5 concepts I must know	Raster grid; RGB bands; CRS; affine transform; NoData/mask.
5 questions judges may ask	Why GeoTIFF? What happens for JPG? What is CRS? How does a pixel map to Earth? Why preserve metadata?

30-second explanation

I own the ingestion layer. I create a consistent RGB model image, mask invalid regions, validate claimed georeferencing, and preserve the original-to-working grid mapping. Person 2 receives aligned RGB; Person 3 receives the CRS, transform, resolution, and mask. I never invent coordinates for PNG/JPG or for a TIFF whose metadata is geographically inconsistent.

2-minute detailed explanation

My module is the foundation of the pipeline. A normal image contains pixel colors, while a GeoTIFF can also contain a CRS and affine transform. I read TIFFs with Rasterio and ordinary images with Pillow, convert band-first arrays into rows × columns × RGB, handle one/two-band fallbacks, and build a valid-pixel mask from NoData and non-finite values. For undeclared black scanner borders I mask only dark regions connected to the edge. Large images are decoded to a bounded 3072-pixel working grid, and the metadata records how that grid maps back to the source. Model RGB uses robust per-channel 2nd/98th percentile normalization, but the original numeric data is not silently overwritten. Most importantly, I validate whether georeferencing is actually plausible. Without a valid CRS and transform, the system remains relative. This protects Person 3’s alignment and prevents a visually plausible but geographically wrong DSM.

Cheat sheet · Person 2 — Monocular relative depth estimation

Prompt	My answer
My module	Infer a unitless, image-aligned near/far surface from one RGB view.
My input	rgb_model.png plus optional valid_mask.npy.
What I do	Load Depth Anything V2 Base; select CUDA/CPU; run single/global+tilled inference; align and blend tiles; validate/mask; create preview and JSON.
My output	relative_depth.npy, depth_metadata.json, relative_depth_preview.png, heightmap.json.
Why it matters	Provides fine scene structure that a coarse DEM cannot resolve, while leaving absolute scale to reference calibration.
Main technologies	PyTorch, CUDA, Transformers, Depth Anything V2, NumPy, Pillow.
5 concepts I must know	Monocular ambiguity; relative inverse depth; learned cues; tiled inference; numerical vs display output.
5 questions judges may ask	How can one image give depth? Is it metres? Which model? Does it know roads/buildings? What fails?

30-second explanation

I run Depth Anything V2 Base to estimate relative inverse depth from one prepared RGB image. The result is aligned to the image but has no physical unit. For large scenes, a global pass establishes consistent structure and overlapping local tiles recover detail without obvious seams. The raw NPY remains unchanged; only the PNG preview is contrast-enhanced. Person 3 uses external references to add approximate metric scale.

2-minute detailed explanation

A single image has no stereo baseline, so depth is fundamentally ambiguous. The pretrained model learns statistical cues such as perspective, texture, occlusion, size, and context; it produces a useful prior, not a measurement. Our default checkpoint is depth-anything/Depth-Anything-V2-Base-hf. It records larger raw values as closer and outputs relative inverse-depth-like values, not metres. I use CUDA with float16 autocast when available and CPU otherwise. If the working image is large, one scene-wide pass supplies the global trend and 768-pixel local tiles with 128-pixel overlap recover roofs, roads, and canopy boundaries. Each local prediction has its scale and offset aligned to the global map; only its fine residual is cosine blended, followed by very light seam smoothing. The output is restored to the Person 1 grid, invalid pixels become NaN, and metadata records the representation, model, device, shape, and tile count. Preview enhancement never changes the NPY used for calibration or measurement.

Cheat sheet • Person 3 — SRTM/GCP calibration and DSM generation

Prompt	My answer
My module	Convert relative depth into an estimated metric geospatial surface when trustworthy references exist.
My input	Valid GeoTIFF grid, relative depth, optional local SRTM/elevation raster, optional GCPs, optional reference DSM.
What I do	Reproject references; extract paired samples; fit linear/inverse/Huber mappings; fuse coarse baseline with calibrated detail; validate and export.
My output	absolute_dsm.tif/.npy, preview_dsm.png, metadata.json, metrics.json.
Why it matters	Monocular depth has structure but no metres; SRTM/GCPs supply the missing absolute anchor.
Main technologies	Rasterio, PyProj, NumPy, SciPy, scikit-learn Huber regression, Matplotlib.
5 concepts I must know	DEM/DSM/nDSM; SRTM; GCP; reprojection; vertical datum.
5 questions judges may ask	Why SRTM? Why not SRTM alone? What is calibration? How accurate is it? Can it replace LiDAR?

30-second explanation

I align SRTM and/or known GCP elevations to the image grid, fit a depth-to-height scale and offset, and create an estimated DSM. SRTM provides coarse absolute elevation; calibrated monocular depth provides estimated fine structure. I export a georeferenced float DSM and record metrics and limitations. Without a valid reference, the website stays in relative mode instead of inventing metres.

2-minute detailed explanation

Person 2 gives a scene-dependent relative signal, so my job is to add an approximate metric reference. I first read the GeoTIFF's CRS, affine transform, footprint, and resolution. For a bounded depth grid I derive a scaled transform that preserves the original extent. SRTM or another elevation raster is reprojected with Rasterio into that exact grid; generic image resize would be wrong because it ignores Earth coordinates. GCPs can be source pixels with elevation or longitude/latitude with elevation. I build valid depth/height pairs and fit linear, inverse, and Huber robust-linear candidates, using cross-validation or a deterministic holdout where possible. If SRTM is present, I use its smoothed values as the low-frequency absolute baseline and add alpha times the calibrated high-frequency depth detail. The result is described as an estimated/fused DSM, not survey truth. Independent reference data can produce MAE, RMSE, correlation, and bias. I also warn about coarse SRTM, semantic errors, and vertical-datum mismatch.

Cheat sheet · Person 4 — Frontend website and user experience

Prompt	My answer
My module	Guide the user from upload to understandable, mode-aware results and 3D exploration.
My input	User files plus FastAPI status/results JSON and file URLs.
What I do	Validate selections; submit multipart job; poll progress; show errors/previews/metadata/downloads; lazy-load the viewer; label uncertainty.
My output	Interactive React experience and API requests.
Why it matters	A technically strong pipeline is unusable—and easy to misinterpret—without clear states, units, warnings, and controls.
Main technologies	React, React Router, Vite, JavaScript, CSS, Fetch/FormData.
5 concepts I must know	Component state; HTTP; multipart upload; polling; relative vs metric UX.
5 questions judges may ask	What does frontend own? How does it call backend? How are errors shown? Why lazy-load? How is uncertainty communicated?

30-second explanation

I built the React/Vite interface. It accepts the image and optional calibration files, starts a backend job, polls progress, and displays previews, metadata, downloads, warnings, and the Three.js terrain. The UI explicitly switches between rel and metre modes so users do not confuse a relative model output with absolute elevation.

2-minute detailed explanation

My responsibility is the full user journey, not only styling. On the Analyze page I accept PNG/JPG/GeoTIFF, validate the extension, allow optional GCP and SRTM uploads only with TIFF input, and send everything in FormData to FastAPI. The backend immediately returns a job ID; the page polls its status, retries a few transient failures, and navigates to the results page only after completion. Results resolve backend URLs for the relative-depth preview, DSM preview, downloads, metadata, texture, and heightmap. The Three.js viewer is lazy-loaded because its bundle is large and only needed on this page. A metadata panel shows dimensions, range, model, calibration source, and border masking. Calibrated outputs are labelled as estimated metres with a validation warning; uncalibrated outputs are rel with an explicit message that absolute elevation is unavailable. This makes the interface part of the scientific integrity of the project.

Cheat sheet • Person 5 — Backend and API integration

Prompt	My answer
My module	Orchestrate files, modules, status, errors, and result discovery behind stable HTTP endpoints.
My input	POSTed image plus optional SRTM/GCP files.
What I do	Stream/validate uploads; isolate jobs; run Persons 1–3 in order; choose calibrated/relative branch; convert heightmap; expose safe results.
My output	Job/status/results JSON and generated-file downloads.
Why it matters	It turns separate scripts into one usable system and protects every data contract between them.
Main technologies	Python, FastAPI, Uvicorn, multipart HTTP, subprocess, JSON, filesystem safety.
5 concepts I must know	API; POST/GET; background job; adapter contract; failure propagation.
5 questions judges may ask	What is an API? Why FastAPI? How are modules called? What if one fails? How does it scale?

30-second explanation

I built the FastAPI bridge. It validates and streams uploads, creates a job, calls preprocessing, depth, and optional calibration in the correct order, prepares the browser heightmap, and exposes status, results, and downloads. Each failure stops safely and returns a clear message. Without calibration, the backend deliberately publishes relative output rather than a false metric DSM.

2-minute detailed explanation

The frontend should not know how to run five different scripts, so my API owns orchestration. POST /api/process accepts image, srtm, and gcp multipart fields. Files are signature-checked, size-limited, streamed in 1 MB chunks, and stored inside a random isolated job. FastAPI returns the job ID and runs the pipeline as a background task. pipeline_runner.py invokes owner modules with sys.executable and argument lists, never a shell string, checks required outputs, captures logs, and updates public progress. Person 1 produces the bounded RGB/metadata, Person 2 produces depth, and Person 3 is called only for a valid georeferenced image with a reference. Otherwise I publish a labelled relative fallback. A converter creates bounded heightmap.json for Person 6. GET status, results, and file routes expose only safe information. The current local queue is appropriate for a hackathon; production would need durable workers, object storage, quotas, authentication, and monitoring.

Cheat sheet · Person 6 — 3D terrain rendering and interactive flythrough

Prompt	My answer
My module	Build an aligned textured terrain, camera system, and honest point-analysis interface in WebGL.
My input	heightmap.json, rgb_model.png, metadata.json.
What I do	Load/validate grid; reduce to ≤ 512 samples; create vertices/triangles/UVs; remove invalid faces; render; navigate; probe raw values.
My output	Overview, free flight, auto flythrough, material modes, relief controls, GPU badge, height/slope HUD.
Why it matters	The 3D layer turns a static raster into an intuitive surface that users can navigate and interrogate.
Main technologies	Three.js 0.179.1, WebGL, Vite, OrbitControls, JavaScript.
5 concepts I must know	Height map; mesh/triangles; UV texture; camera translation/rotation; display exaggeration vs data.
5 questions judges may ask	How is mesh built? Why triangles/downsample? How is texture aligned? Are heights real? Does smoothing change values?

30-second explanation

I convert the height grid into a Three.js mesh, map the aligned RGB image onto it, and provide overview, free-flight, and automatic camera modes. The viewer downsamples large grids for performance, removes invalid-border triangles, and uses high-quality texture filtering and soft shadows. A centre reticle reports the unmodified height, slope, source pixel, and map coordinate, with rel or metre labels depending on calibration.

2-minute detailed explanation

Person 6 receives a compact JSON grid and aligned texture. Each row/column sample becomes a vertex, neighboring valid samples become triangles, and UV coordinates map the original image to the same orientation. I preserve physical X/Z aspect from pixel resolution; geographic degree grids receive an approximate local metre conversion. Metric DSM values use their elevation difference directly, while relative values are robustly clipped and conservatively scaled because they have no physical vertical unit. The renderer limits the long edge to 512 samples, uses bilinear reduction, discards triangles across invalid pixels, adds anisotropic mipmapped texture, ACES tone mapping, soft shadows, and a high-performance WebGL hint. Users can orbit, fly with W/S and A/D strafing, rise/fall, or follow an automatic path. The centre ray probes the original numeric field and calculates local slope; smoothing and relief affect display only. The GPU badge tells us whether the browser actually selected NVIDIA, Intel, or software rendering.

Appendix A • Glossary

Term	Meaning
Absolute elevation	Height referenced to a specified vertical datum; in this prototype, estimated only after calibration.
Affine transform	Six coefficients that map raster pixel coordinates to map coordinates.
API	A defined interface through which software components exchange requests and responses.
Bias	Mean predicted elevation minus reference elevation.
Calibration	Fitting a relationship between relative model values and known reference heights.
CRS	Coordinate Reference System: rules that give map coordinates Earth meaning.
CUDA	NVIDIA's GPU-computing platform used here for PyTorch inference.
DEM	General term for a raster of elevation values; product meaning must be checked.
DSM	Digital Surface Model: elevation of the sensed upper surface, potentially including buildings and vegetation.
DTM	Digital Terrain Model: usually a bare-earth surface.
GCP	Ground Control Point with known location/elevation used as a calibration anchor.
GeoTIFF	TIFF raster containing geospatial reference metadata.
Height map	2D grid whose cells store height or relative-height samples for a mesh.
Inverse depth	A representation where larger values often mean closer, opposite to ordinary distance.
MAE	Mean Absolute Error: average magnitude of vertical errors.
Mesh	Vertices connected into faces, here triangles, forming a 3D surface.
NoData	A marker or mask identifying cells without valid observations.
nDSM	Normalized DSM: surface elevation minus a reliable ground terrain model.
Orthometric height	Height relative to a gravity-based geoid-like sea-level reference.
Raster	A regular row-and-column grid of cells; bands store one or more values per cell.
Relative depth	Scene-dependent near/far structure without a physical scale.
Reprojection	Mapping raster samples between coordinate systems using geospatial metadata.
RMSE	Root Mean Square Error: error metric that penalizes large deviations.
SRTM	Shuttle Radar Topography Mission elevation data used as a coarse reference.
Texture / UV	The image and coordinates that map its pixels onto a 3D surface.
Vertical datum	Reference surface that defines the zero for elevation.
WebGL	Browser graphics API used by Three.js to render on the GPU.

Appendix B • Verified repository file map

Person	Main location	Key files
1	depthwizard_person1	main.py; config.py; src/geotiff.py, preprocessing.py, coordinates.py, output.py
2	depthwizard_person2	config.py; src/depth_model.py, inference.py, postprocessing.py, output.py
3	depthwizard_person3	main.py; src/alignment.py, calibration.py, fusion.py, validation.py, export.py; examples/
4	depthwizard_person4	src/pages/*.jsx; src/components/*.jsx; src/services/realApi.js; src/styles/global.css
5	depthwizard_person5	main.py; api/routes.py; config.py; file_manager.py; pipeline_runner.py
6	depthwizard_person6	src/main.js, terrain.js, dataLoader.js, firstPerson.js, flythrough.js; tools/prepare_heightmap.py
Shared	repository root	README.md; start_depthwizard.cmd; start_backend.*; start_frontend.*; open_gpu_viewer.*; test_all.ps1

Default configuration snapshot

Setting	Current default
Depth model	depth_anything_v2_base / depth-anything/Depth-Anything-V2-Base-hf
Model working-grid long edge	3072 pixels
Local inference tile / overlap	768 / 128 pixels
Browser terrain long edge	512 samples
Upload limit	500 MB
Pipeline stage timeout	1800 seconds
Frontend / backend	React + Vite / FastAPI + Uvicorn
3D library	Three.js 0.179.1

Appendix C • References and further reading

The repository source code and test suite are the authority for implementation details in this guide. The following primary or official sources support model, geospatial, elevation, API, and rendering concepts.

Source	Use in this guide	URL
[1] Depth Anything V2 paper	Yang et al., arXiv:2406.09414	https://arxiv.org/abs/2406.09414
[2] Depth Anything V2 Base model card	Hugging Face model repository and usage notes	https://huggingface.co/depth-anything/Depth-Anything-V2-Base-hf
[3] SRTM product specifications	USGS EROS: WGS84, EGM96, metre units, 1 arc-second ≈ 30 m	https://www.usgs.gov/centers/eros/science/usgs-eros-archive-digital-elevation-shuttle-radar-topography-mission-srtm
[4] SRTM Collection User Guide	NASA LP DAAC/USGS Version 3 guide	https://lpdaac.usgs.gov/documents/179/SRTM_User_Guide_V3.pdf
[5] Rasterio reprojection	Official Rasterio documentation	https://rasterio.readthedocs.io/en/stable/topics/reproject.html
[6] FastAPI background tasks	Official FastAPI tutorial	https://fastapi.tiangolo.com/tutorial/background-tasks/
[7] Three.js WebGLRenderer	Official renderer options, GPU capability, tone mapping, shadows	https://threejs.org/docs/pages/WebGLRenderer.html

Final team message

DepthWizard is strongest when the team presents it as an integrated, evidence-aware prototype: relative geometry from a single image; metric anchoring only from trustworthy references; geospatial metadata preserved; numerical and visual transformations separated; and uncertainty explained directly. That honesty makes the engineering more credible, not less ambitious.